

PLAN DE MIGRATION & MODERNISATION

Smart BERRY — Berry Good Farms Dashboard

Refonte architecturale — app.jsx monolithe (68 153 lignes)

Version 1.1 — 21/08/2026

Statut : Plan directeur validé, prêt pour exécution

1. Résumé exécutif

L'application Smart BERRY est une SPA React de 68 153 lignes concentrée dans un monolithe public/app.jsx. Elle gère 35+ domaines métier (RH, Qualité, Finance, Achats, Agronomie, etc.) avec 100+ composants React, 50+ utilitaires et 30+ objets de configuration.

Problèmes critiques

- Maintenance impossible : toute modification touche un fichier de 5,3 Mo
- Tests impossibles : aucun composant isolé, pas d'imports ES
- Performance dégradée : chargement complet de 5,3 Mo au démarrage
- Risque de régression élevé : couplage fort entre tous les composants
- Pas de code splitting : tout le code est chargé même si l'utilisateur n'utilise qu'un seul onglet

Objectifs de la migration

- Réduire la complexité : aucun fichier > 2 000 lignes
- Activer les tests : composants isolés, testables avec React Testing Library
- Améliorer les performances : code splitting → chargement initial < 500 Ko
- Faciliter la collaboration : modules indépendants, PR focalisées
- Préparer l'avenir : architecture scalable pour 5+ années

Métriques de succès

Métrique	Avant	Cible
Taille fichier principal	5,3 Mo	< 200 Ko
Temps de chargement initial	~3-5 s	< 1 s
Couverture de tests	0 %	> 60 % des composants
Lignes max. par fichier	68 153	< 2 000
Nombre de modules	1	11+

2. État des lieux — Architecture actuelle

Le fichier `public/app.jsx` contient l'intégralité de l'application frontend. Cette architecture monolithique pose des défis majeurs de maintenance, de testabilité et de performance.

Analyse quantitative

Catégorie	Quantité	Lignes est.	Impact
Composants React	100+	~50 000	Critique
Tableaux de bord	12	~8 000	Haute complexité
Formulaires de saisie	25	~12 000	Logique métier dense
Vues de données	40	~18 000	Requêtes Firestore imbriquées
Utilitaires	50+	~5 000	Important
Configuration	30+	~3 000	Important
Charts SVG	4	~400	Faible impact

Points de douleur identifiés

Critiques :

- Impossible à tester unitairement : aucun export, pas d'import ES
- Recherche/navigation lente : VSCode ralentit sur les fichiers > 5 Mo
- Git diffs énormes : toute PR touche le même fichier
- Cache navigateur invalidé en bloc : 1 ligne modifiée = 5,3 Mo à re-télécharger
- Pas de Hot Module Replacement (HMR) : rechargement complet à chaque modification

Importants :

- Couplage fort : impossible de modifier un composant sans risquer d'en casser un autre
- Duplication de code : mêmes patterns répétés sans réutilisation
- Pas de séparation des responsabilités : UI + logique métier + accès données mélangés
- Performance dégradée : chargement complet même si l'utilisateur n'utilise que le module RH

3. Architecture cible

L'architecture cible repose sur une organisation modulaire avec 11 modules métier indépendants, des bibliothèques partagées réutilisables, et un système de code splitting pour optimiser le chargement.

Principes directeurs

- Séparation des responsabilités : Pages, Composants, Hooks, Utils, API, Contexts
- Code splitting & lazy loading : chargement à la demande par module
- Testabilité : tous les composants exportés et testables unitairement
- Réutilisabilité : bibliothèques partagées (charts, utils, hooks)
- Pragmatisme : structure adaptée à la taille de chaque module

Règle de structure par taille

Principe : commencer simple, complexifier seulement si nécessaire. Éviter la sur-ingénierie.

Taille module	Nb fichiers	Structure	Sous-répertoires
Petit	< 10	PLATE	Aucun (tous à la racine)
Moyen	10-20	GROUPEE	Si 3+ fichiers d'un même type
Grand	> 20	COMPLÈTE	pages/, components/, hooks/

- Structure PLATE (petit module — ex. Sécurité, 5 pages) : tous les fichiers à la racine du module. Simple, direct, moins de clics pour naviguer. Parfait pour 5-10 fichiers.
- Structure GROUPEE (module moyen — ex. RH, 6 pages) : pages à la racine, sous-dossiers shared/ et hooks/ seulement si 3+ fichiers. Équilibre simplicité et organisation.
- Structure COMPLÈTE (grand module — ex. Qualité, 14 pages) : organisation complète pages/, components/, hooks/. Évite d'avoir 30+ fichiers à la racine.

4. Modules fonctionnels proposés

Module	Pages	Structure	Priorité
Caisse	3	PLATE	P0 (Fait)
Sécurité	5	PLATE	P5 (Sprint 7)
Récolte	4	PLATE	P1 (Sprint 1)
RH & Paie	6	GROUPÉE	P1 (Sprint 1)
Magasin	9	GROUPÉE	P3 (Sprint 4)
Technique	8	GROUPÉE	P4 (Sprint 6)
Admin	10	GROUPÉE	P5 (Sprint 7)
Qualité	14	COMPLÈTE	P2 (Sprint 2-3)
Agronomie	14	COMPLÈTE	P4 (Sprint 5-6)
Finance	15	COMPLÈTE	P2 (Sprint 2-3)
Achats	15	COMPLÈTE	P3 (Sprint 4-5)

Total : 103 pages (composants majeurs)

Détail des modules

Module RH & Paie

Pages (6) :

DashboardTab, PointageTab, PointageValidationTab, PaieTab, PrimesTab, EquipesTab

Composants clés (4) :

WorkerCard, PointageTable, PaieCalculator, PrimeForm

Utilitaires (3) :

calculatePaie(), validatePointage(), calculatePrime()

Module Récolte & Production

Pages (4) :

RecolteTab, CoutRecolteTab, HorsRecolteTab, HorsRecolteSuiviTab

Composants clés (3) :

RecolteForm, RendementChart, CoutAnalysis

Utilitaires (2) :

calculateRendement(), calculateCoutRecolte()

Module Qualité

Pages (14) :

QualiteDashboardTab, QualitePFQInterneTab, QualiteBonsApportTab, QualiteInspectionsTab, QualiteHistoriqueTab, QualiteProductionTab, QualiteBrixTab, QualiteMarcheLocalTab, QualiteExpeditionsTab, QualiteReconciliationTab, QualiteLiquidationsTab, QualiteEcartTab, QualiteValidationBonsTab, DQRDailyTab

Composants clés (4) :

PFQTable, InspectionForm, DefectChart, LiquidationRecap

Utilitaires (3) :

calculatePFQ(), detectDefects(), reconcileLiquidations()

Module Finance

Pages (15) :

FinDashboardTab, FinCATab, FinTresorerieTab, FinCarburantTab, FinPlantsTab, FinTelecomTab, FinOjraTab, FinStockTab, FinLiquidationsTab, FinMarcheLocalTab, FinBDCTab, FinCodesAnalytiquesTab, FinVirementsTab, FinFacturesTab, FinPaiementsTab

Composants clés (4) :

TresoForecast, BudgetChart, FactureTable, PaiementForm

Utilitaires (3) :

calculateTreso(), forecastCA(), reconcileBudget()

5. Plan de migration par phases

Plan révisé : livraison en 20 jours ouvrés maximum ($\approx$ 4 semaines, $\approx$ 28 jours calendaires — sous le plafond de 30 jours), par un développeur senior seul assisté de Claude Code, sur le périmètre non-régression défini en section 9 (aucune modification d'interface ; les correctifs éventuels sont traités en régie séparément). Le plan en 6 phases sur 165 jours ouvrés décrit plus haut correspond au scénario équipe (section 8, à titre de référence) ; il n'est plus le scénario retenu.

Phase	Durée	Contenu	Livrable clé
Phase 0 — Setup & bibliothèques	4 j	Vite/TS/Vitest/CI + extraction Charts/API/Utils/Constants	Socle technique + libs partagées testées
Phase 1 — Modules simples & moyens	2 j	Sécurité, Récolte, RH & Paie, Magasin, Technique, Admin (61 pages)	6 modules migrés, non-régression validée
Phase 2 — Modules complexes	4 j	Qualité, Agronomie, Finance, Achats (58 pages)	4 modules migrés, calculs/indicateurs vérifiés identiques
Phase 3 — Intégration transverse	3,5 j	Feature flags, tests croisés entre modules, validation staging	Dual mode opérationnel sur l'ensemble
Phase 4 — Finalisation	4,5 j	Perf, E2E, documentation, déploiement prod, monitoring	Production ready
Marge de sécurité (15 %)	$\sim$ 2 j	Aléas, allers-retours de validation	—

Total : 20 jours ouvrés ($\approx$ 158 h) — voir le détail chiffré en section 9.

Phase 0 : Setup & bibliothèques partagées (4 jours)

- Installer Vite, configurer TypeScript (mode permissif), Vitest + React Testing Library, ESLint/Prettier, CI de base
- Extraire Charts, API/Cache, Utils, Constants (bibliothèques partagées, $\sim$ 3 300 lignes)

Critère de succès : l'app shell s'affiche, les libs extraites passent leurs tests, l'app fonctionne à l'identique

Phase 1 : Modules simples & moyens (2 jours)

- Sécurité, Récolte, RH & Paie, Magasin, Technique, Admin — 61 pages, structure PLATE/GROUPEE

Critère de succès : chaque page migrée se comporte à l'identique de l'existant (non-régression)

Phase 2 : Modules complexes (4 jours)

- Qualité, Agronomie, Finance, Achats — 58 pages, structure COMPLÈTE, calculs et règles métier denses

Critère de succès : montants et indicateurs strictement identiques à l'existant, sur échantillon de contrôle

Phase 3 : Intégration transverse (3,5 jours)

- Câblage des feature flags, tests croisés entre modules, validation en environnement de staging

Critère de succès : dual mode bascule sans régression visible sur l'ensemble des 11 modules

Phase 4 : Finalisation (4,5 jours)

- Optimisations de performance, suite de tests E2E, documentation, déploiement en production, mise en place du monitoring

Critère de succès : Lighthouse > 90, 0 régression détectée en staging, déploiement prod réussi

6. Recommandations de modernisation

1. TypeScript — Migration progressive

Approche en 3 étapes :

- Phase 1 : JSDoc strict sur tous les helpers extraits
- Phase 2 : convertir les nouveaux modules en .tsx
- Phase 3 : activer "strict": true progressivement

2. CI/CD — GitHub Actions

Pipeline automatisé :

- Lint → Test → Build → Deploy
- Déploiements preview automatiques sur chaque PR
- Déploiement prod automatique sur main
- Tests E2E avant déploiement

3. Tests automatisés — Pyramide

Type	Nombre	Outils	Couverture cible
Unitaires	300+	Vitest + RTL	> 80 %
Intégration	20-30	Vitest	Workflows critiques
E2E	5-10	Playwright	User journeys

4. Performance

- Code splitting granulaire : préchargement au hover
- Memoization agressive : React.memo + useMemo
- Virtualisation des tables longues : @tanstack/react-virtual
- Service Worker avec Workbox : cache intelligent

Métriques cibles :

- Lighthouse Performance : > 90
- First Contentful Paint (FCP) : < 1,5 s
- Time to Interactive (TTI) : < 3 s
- Total Blocking Time (TBT) : < 300 ms

7. Stratégie de migration sans régression

Approche progressive

Migration module par module, en gardant l'ancien code fonctionnel jusqu'à validation complète. Dual mode activable via feature flags Firebase Remote Config.

```
// Dual Mode : ancien + nouveau en parallèle

const USE_NEW_MODULES = getFeatureFlag('new_modules_enabled');

function App() {
  return (
    <Routes>
      {USE_NEW_MODULES ? (
        <Route path="/rh/*" element={<RHModuleNew />} />
      ) : (
        <Route path="/*" element={<LegacyApp />} />
      )}
    </Routes>
  );
}
```

Tests de non-régression

- Tests fonctionnels par module
- Tests par profil utilisateur
- Tests performance (temps de chargement, memory leaks)
- Snapshot tests pour les calculs critiques
- Visual regression tests (Playwright)

Rollback plan — 3 niveaux

- Niveau 1 — Feature Flag (instantané) : retour à l'ancien code en < 1 min
- Niveau 2 — Déploiement précédent (5 min) : firebase hosting:clone
- Niveau 3 — Git Revert (10 min) : git revert + CI/CD redéploie

8. Roadmap d'exécution

Roadmap révisée pour le scénario retenu (développeur senior + Claude Code, section 9) : 20 jours ouvrés, démarrage le 1er septembre 2026, livraison le 28 septembre 2026 (28 jours calendaires, sous le plafond de 30 jours).

Timeline détaillée

Phase	Durée	Dates estimées	Jalon critique
Phase 0 — Setup & libs	4 j	01 sept → 04 sept	Socle technique + libs partagées testées
Phase 1 — Modules simples/moyens	2 j	07 sept → 08 sept	6 modules migrés (61 pages)
Phase 2 — Modules complexes	4 j	09 sept → 14 sept	4 modules migrés (58 pages)
Phase 3 — Intégration transverse	3,5 j	15 sept → 17 sept	Dual mode opérationnel
Phase 4 — Finalisation	4,5 j	18 sept → 24 sept	Production ready
Marge de sécurité	~2 j	25 sept → 28 sept	Absorption des aléas

Ressources nécessaires

Rôle	Charge	Durée	Total
Développeur senior full-stack + Claude Code	Temps plein	20 j ouvrés (~1 mois)	≈ 158 h

Ressource unique : le scénario équipe (Lead Dev + 2 Dev Frontend + Dev Backend mi-temps + QA mi-temps, 23,5 mois-homme sur 7 mois) reste documenté en section 9 à titre de comparaison, mais n'est plus le scénario retenu.

KPIs de succès

KPI	Avant	Cible	Mesure
Taille fichier principal	5,3 Mo	< 200 Ko	Bundle analyzer
Temps chargement initial	3-5 s	< 1 s	Lighthouse
Lignes max par fichier	68 153	< 2 000	Analyse statique
Nombre de modules	1	11+	Structure dossiers
Non-régression fonctionnelle	—	100 % des pages migrées	Recette manuelle + tests croisés
Lighthouse Performance	~60-70	> 90	Lighthouse CI
Bugs critiques en prod	Variable	0	Sentry

9. Estimation de charge — Développeur senior + Claude Code

Cette section décompose la charge poste par poste (page par page). Un développeur senior full-stack (React/Node/Firebase) travaille seul avec Claude Code comme copilote pour l'extraction de code et la génération de tests ; il assure lui-même la revue, la validation métier et le déploiement progressif.

Hypothèses de chiffrage

- 1 développeur senior, épaulé par Claude Code pour l'extraction/refactoring mécanique et la génération de tests
- Journée de travail effective : 8 heures
- Le forfait couvre un travail de non-régression : extraction du code existant vers la nouvelle architecture modulaire, sans modification de l'interface ni de la logique métier — l'objectif est un comportement strictement identique à l'existant
- Toute anomalie nécessitant une modification (bug préexistant révélé, ajustement d'interface, logique manquante) sort du forfait et relève de la clause de surcharge ci-dessous
- La charge exclut : ateliers de cadrage additionnels, formation approfondie d'une équipe existante

Méthode : temps par page selon la complexité

Le temps par page suit la même classification que la section 3 (structure PLATE / GROUPÉE / COMPLÈTE). En non-régression stricte, ce temps couvre l'extraction assistée par Claude Code et la vérification que la page migrée se comporte à l'identique — pas une réécriture.

Structure	Charge / page	Justification
PLATE	10 min	Composants simples (listes, formulaires courts) : vérification visuelle et fonctionnelle rapide
GROUPÉE	20 min	Logique métier modérée (paie, stock) : quelques cas de test à repasser
COMPLÈTE	30 min	Calculs et règles métier denses (finance, qualité) : les montants/indicateurs doivent rester rigoureusement identiques

Charge par module

Module	Pages	Structure	Temps/page	Charge
Caisse	3	—	—	0 h (déjà fait — P0)
Sécurité	5	PLATE	10 min	0,8 h
Récolte	4	PLATE	10 min	0,7 h
RH & Paie	6	GROUPÉE	20 min	2,0 h
Magasin	9	GROUPÉE	20 min	3,0 h
Technique	8	GROUPÉE	20 min	2,7 h
Admin	10	GROUPÉE	20 min	3,3 h
Qualité	14	COMPLÈTE	30 min	7,0 h

Module	Pages	Structure	Temps/page	Charge
Agronomie	14	COMPLÈTE	30 min	7,0 h
Finance	15	COMPLÈTE	30 min	7,5 h
Achats	15	COMPLÈTE	30 min	7,5 h

Sous-total pages (100 pages restantes) : ≈ 41,5 h

Blocs transverses (indépendants du nombre de pages)

Ces blocs sont resserrés par rapport à la version précédente pour tenir dans le plafond de 20 jours ouvrés — c'est le principal levier de réduction, les blocs transverses étant le poste dominant (voir la remarque de la section précédente).

Bloc	Charge	Contenu
Setup	12 h	Vite, TypeScript, Vitest, ESLint, CI de base
Extraction Utils/API/Charts/Constants	20 h	Bibliothèques partagées (~3 300 lignes)
Intégration transverse & non-régression	28 h	Feature flags, tests croisés entre modules, validation staging
Finalisation	36 h	Perf, tests E2E, documentation, déploiement prod, monitoring

Sous-total blocs transverses : 96 h

Total

Sous-total (41,5 h + 96 h = 137,5 h), majoré de 15 % pour aléas :

TOTAL NON-RÉGRESSION : ≈ 158 h ≈ 20 jours ouvrés (≈ 28 jours calendaires — sous le plafond de 30 jours)

Pour tenir ce plafond, les blocs transverses ont été resserrés de 112 h à 96 h (setup, extraction utils, intégration, finalisation), sans réduire le périmètre fonctionnel (les 11 modules et 100 pages restent couverts). Cette compression laisse moins d'absorption naturelle des imprévus qu'une version à 22 jours ; la clause de régie ci-dessous devient donc le mécanisme principal de gestion des écarts, plutôt qu'une marge intégrée confortable.

Clause de surcharge (régie)

Le forfait de 158 h couvre l'extraction et la vérification de non-régression, sans modification. Si une page révèle un besoin de correction (bug préexistant, ajustement d'interface, logique manquante ou erronée), les travaux correspondants sont traités en régie, au tarif horaire convenu, après validation préalable du client sur le périmètre et le temps estimé de l'intervention. Aucune modification n'est engagée sans cet accord préalable.

Comparaison avec l'estimation en équipe (section 8)

L'estimation en équipe historique totalisait 23,5 mois-homme, soit environ 3 760 heures cumulées. Le forfait non-régression solo + Claude Code à 20 jours (≈ 158 h) représente un volume d'heures cumulées environ 24 fois inférieur — à condition que l'hypothèse « aucune modification » se vérifie. C'est un chiffrage plancher, avec une marge d'aléas volontairement resserrée pour tenir le plafond de durée demandé.

Proposition tarifaire

Sur la base du forfait non-régression à 20 jours (≈ 158 h) :

Tarif horaire (MAD)	Total forfait (158 h)
400 MAD/h	63 200 MAD
600 MAD/h	94 800 MAD
800 MAD/h	126 400 MAD

Formule : Total = 158 h × Tarif horaire retenu. Le paiement peut être jalonné par phase (voir section 5 et 8). Les heures de régie pour modifications (clause ci-dessus) sont facturées en sus, au même tarif horaire, sur la base du temps réellement passé et validé.

10. Conclusion

Cette migration représente un investissement majeur mais indispensable pour la pérennité de Smart BERRY. Les bénéfices attendus sont considérables.

Bénéfices immédiats

- Maintenance facilitée : code modulaire, fichiers < 2 000 lignes
- Tests possibles : 400+ tests unitaires + E2E
- Performance améliorée : chargement initial < 1 s (vs 3-5 s)
- Collaboration efficace : PR focalisées, pas de conflits git

Bénéfices long terme

- Scalabilité : ajout de nouveaux modules facile
- Qualité du code : TypeScript + linting + code reviews
- Onboarding rapide : nouveaux devs autonomes en 1 semaine (vs 1 mois)
- Dette technique maîtrisée : refactoring continu possible

Risques maîtrisés

- Régression : dual mode + feature flags + rollback plan
- Délais : sprints incrémentaux + MVP par module
- Performance : monitoring + optimisations + tests Lighthouse

Recommandation

Démarrer la migration dès que possible. Chaque mois de retard augmente la dette technique et le risque de blocage sur les futurs sprints.

11. Annexes

Glossaire

- SPA : Single-Page Application — application web qui charge une seule page HTML
- Code Splitting : division du code en morceaux chargés à la demande
- Lazy Loading : chargement différé (à l'utilisation)
- Tree Shaking : suppression du code inutilisé au build
- Memoization : mise en cache du résultat d'un calcul coûteux
- JSDoc : annotations JS pour documentation/typage
- TypeScript : superset de JS avec typage statique
- Vitest : framework de tests unitaires (fork de Jest pour Vite)
- Playwright : framework de tests E2E multi-navigateurs
- Lighthouse : outil d'audit de performance web (Google)

Checklist finale

Avant de démarrer

- ☐ Budget validé (23,5 mois-homme)
- ☐ Équipe constituée
- ☐ Environnement staging disponible
- ☐ Accès Firebase + GitHub + Sentry
- ☐ Backup complet des données de production

Phase 0 (Setup)

- ☐ Vite build fonctionne
- ☐ Tests unitaires opérationnels
- ☐ Linting automatique
- ☐ Structure de dossiers créée

Phase 1 (Utilitaires)

- ☐ Charts extraits
- ☐ API/Cache extraits
- ☐ Utils extraits
- ☐ Constants extraits
- ☐ 50+ tests verts

Phase 6 (Finalisation)

- ☐ Optimisations performance
- ☐ Documentation complète
- ☐ Tests E2E complets
- ☐ Lighthouse > 90
- ☐ Déploiement prod réussi

- ☐ Monitoring actif
- ☐ Équipe formée
- ☐ Ancien code supprimé

Document préparé par : Berry Good Farms — Technical Team

Date : 21/08/2026

Version : 1.1 (Approche Pragmatique)

Statut : Plan directeur validé, prêt pour exécution